

C++ Object-Oriented Programming: The Interview Playbook

A professor's rigor meets a recruiter's checklist — the mechanics interviewers actually probe, from vtable layout to the diamond problem, with academic definitions and production-grade code.

Level: New-grad → Staff Engineer **Standard:** C++11 through C++20 idioms **Structure:** 9 categories, 34 worked questions

Grounded in: [GeeksforGeeks](#), [LeetCode Discuss](#), [Glassdoor](#), *Effective C++*

01 The Four Pillars of OOP, in C++ Terms

Object-oriented programming organizes a system around *objects* — units that bundle state and behavior — rather than around free-standing procedures. Every syllabus and interview loop converges on four pillars: **Encapsulation**, **Abstraction**, **Inheritance**, and **Polymorphism**.

What separates a junior answer from a senior one is naming the exact C++ language feature that implements each pillar, and knowing where the pillar's guarantees end.

Q1. What are the four pillars of OOP, and what specific C++ mechanism implements each?

FOUNDATIONAL

DEFINITION

Encapsulation bundles data and the operations on that data into one unit and restricts direct access to internal state. Abstraction exposes only the essential interface of a type while hiding implementation detail. Inheritance lets one type acquire the interface and implementation of another. Polymorphism lets code invoke behavior through a common interface while the concrete behavior varies by the underlying type.

In C++ specifically:

```
class BankAccount {
public:
    explicit BankAccount(double openingBalance) : balance_(openingBalance) {}

    void deposit(double amount) {
        if (amount ≤ 0) throw std::invalid_argument("deposit must be positive");
        balance_ += amount;           // Encapsulation: invariant protected
    }
    double balance() const { return balance_; } // controlled read access

private:
    double balance_;                 // hidden representation
};
```

Encapsulation → `private` / `protected` access specifiers plus member functions that guard invariants (here, balance can never go negative through deposit alone). **Abstraction** → abstract base classes and pure virtual functions that publish a contract without exposing how it is fulfilled. **Inheritance** → the `class Derived : public Base` syntax. **Polymorphism** → `virtual` functions for runtime dispatch, and templates/overloading for compile-time dispatch.

Why asked: Filters candidates who memorized definitions from those who can map each term onto a concrete keyword or idiom.

Q2. Encapsulation and abstraction sound similar – how do they actually differ?

FOUNDATIONAL

Encapsulation is a **mechanism**: it controls *who* can touch *which* data, using access specifiers. Abstraction is a **design goal**: it decides *what* the outside world needs to know at all. You can encapsulate without abstracting well (private fields with getters/setters for every one of them just re-exposes the representation), and you can design a clean abstraction that is technically not very encapsulated (a struct of public fields can still represent a well-abstracted, minimal interface if that's genuinely the whole contract).

```
class Shape {                                // Abstraction: callers only need area()
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
public:
    explicit Circle(double r) : radius_(r) {} // Encapsulation: radius_ hidden
    double area() const override { return 3.14159265 * radius_ * radius_; }
private:
    double radius_;
};
```

Why asked: A classic "explain the difference" trap that separates rote learners from engineers who reason about design intent.

Q3. Distinguish compile-time polymorphism from runtime polymorphism, with the mechanism behind each.

INTERMEDIATE

DEFINITION

Compile-time (static) polymorphism is resolved by the compiler using only the static types visible at the call site — function overloading, operator overloading, and templates. Runtime (dynamic) polymorphism is resolved using the object's actual dynamic type at execution time, via virtual dispatch.

```
void print(int x)    { std::cout << "int: "    << x; }
void print(double x) { std::cout << "double: " << x; } // overload resolved at compile time

struct Animal { virtual std::string speak() const { return "..."; } };
struct Dog : Animal { std::string speak() const override { return "Woof"; } };

void announce(const Animal& a) {
    std::cout << a.speak();    // resolved at RUNTIME via vtable, based on a's dynamic type
}
```

The practical distinction interviewers want: overload resolution can never change behavior based on data computed at runtime — it is a purely syntactic, compile-time decision on argument types. Virtual dispatch is the only one of the two that lets the exact same call site produce different behavior depending on which concrete object shows up at runtime.

Why asked: Confirms you understand that "polymorphism" in C++ is not one feature — it's two independent mechanisms with very different costs and guarantees.

Q4. C++ is described as favoring "zero-overhead abstraction." How does that philosophy shape its OOP features?

ADVANCED

Bjarne Stroustrup's zero-overhead principle states: you don't pay for what you don't use, and what you do use, you couldn't hand-code any better. This is why C++ never made every member function virtual by default (unlike Java) — a non-virtual call is a direct, statically-resolved jump with no indirection, while a virtual call costs one pointer dereference through the vtable. You opt into dynamic dispatch per function with the `virtual` keyword, and opt into compile-time polymorphism for free via templates, which are fully resolved and often fully inlined before the program ever runs.

Why asked: A senior-level question probing whether you understand

why C++'s OOP model looks different from Java/Python's, not just that it does.

02 Classes, Structs & Access Control

Classes are C++'s user-defined types. The rules governing visibility, defaults, and special "escape hatches" like `friend` and `static` are a dense, fast-moving portion of most screening interviews because they are easy to test and easy to get subtly wrong.

Q5. What is the actual difference between `class` and `struct` in C++?

FOUNDATIONAL

ASPECT	STRUCT	CLASS
Default member access	public	private
Default inheritance	public	private
Convention	Passive data aggregate (POD-like)	Type with invariants and behavior

Every other capability — constructors, virtual functions, templates, inheritance — is identical between the two keywords. The distinction is purely the two defaults above; the "struct = data, class = behavior" split is convention, not a compiler-enforced rule.

Why asked: A five-second sanity check early in phone screens; interviewers watch for candidates who only know "struct is public" but miss the inheritance-default half.

Q6. Walk through how `public`, `protected`, and `private` inheritance change a base member's accessibility in the derived class.

INTERMEDIATE

BASE MEMBER ACCESS	PUBLIC INHERITANCE	PROTECTED INHERITANCE	PRIVATE INHERITANCE
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

Base-class `private` members are never directly reachable from a derived class regardless of the inheritance mode — they can only be manipulated indirectly through the base class's own public/protected member functions. Interviewers usually follow up by asking when you'd ever choose `private` inheritance: it models "implemented in terms of" rather than "is-a," and is largely superseded in modern style by composition (a private member of the base type) unless you specifically need access to protected members or to override virtual functions.

Why asked: Tests whether you can reason about a matrix of rules rather than just the common public-inheritance case.

Q7. What is a `friend` function or class, and why is it controversial from an encapsulation standpoint?

INTERMEDIATE

A `friend` declaration grants an external function or class access to a type's private and protected members, bypassing normal encapsulation. It is a deliberate, explicit, one-way grant — the class chooses its friends; nothing can force friendship on it. The canonical justified use is operator overloading where the left-hand operand isn't the class itself, such as `operator<<` for stream output:

```
class Vector2D {
public:
    Vector2D(double x, double y) : x_(x), y_(y) {}
    friend std::ostream& operator<<(std::ostream& os, const Vector2D& v);
private:
    double x_, y_;
};

std::ostream& operator<<(std::ostream& os, const Vector2D& v) {
    return os << "(" << v.x_ << ", " << v.y_ << ")"; // needs private access, isn't a member
}
```

The critique: `friend` widens the effective boundary of a class beyond its own member list, so two files must be read together to understand the true invariant surface. Used sparingly (operator overloads, tightly-coupled builder/factory pairs, unit test fixtures) it's idiomatic; used broadly it's a code smell indicating the class boundary was drawn in the wrong place.

Why asked: Senior interviewers use this to gauge design judgment, not just syntax recall.

Q8. How do `static` data members and member functions differ from instance members? Trick: does a static member function have a `this` pointer?

FOUNDATIONAL

```
class Counter {
public:
    Counter() { ++instanceCount_; }
    ~Counter() { --instanceCount_; }
    static int liveInstances() { return instanceCount_; } // no `this`, class-wide
private:
    static int instanceCount_; // one copy shared by ALL Counter objects
};

int Counter::instanceCount_ = 0; // out-of-class definition required (pre-C++17)
```

A static data member exists once per class, not once per object — every instance shares the same storage. A static member function is not bound to any particular instance, so it has **no `this` pointer** and cannot access non-static members directly, nor can it be declared `virtual` or `const` (there's no object to be const). Since C++17, `inline static` members can be defined directly in the class body without a separate out-of-line definition.

Why asked: "Can a static function be virtual?" is a frequent oral rapid-fire question because it forces you to justify the answer from first principles (no receiver object → no vtable lookup possible).

03 Object Lifecycle: Constructors, Destructors & RAII

This is the category where C++ interviews diverge hardest from Java/Python OOP interviews. Because C++ has deterministic destruction and no garbage collector, lifecycle management — RAII, the Rule of Five, and virtual destructors — is treated as core OOP knowledge, not a separate "systems" topic. Much of this section traces directly back to Scott Meyers' *Effective C++*, still the most-cited reference in senior C++ loops.

Q9. What is RAII, and why is it called C++'s defining resource-management idiom?

FOUNDATIONAL

DEFINITION

Resource Acquisition Is Initialization: bind the lifetime of any resource (memory, file handle, mutex, socket) to the lifetime of a stack-allocated object. The constructor acquires the resource; the destructor releases it — automatically, deterministically, and exception-safely, no matter how the enclosing scope is exited.

```
class FileHandle {
public:
    explicit FileHandle(const char* path) : fp_(std::fopen(path, "r")) {
        if (!fp_) throw std::runtime_error("failed to open file");
    }
    ~FileHandle() { if (fp_) std::fclose(fp_); }    // runs on every exit path, incl. exceptions

    FileHandle(const FileHandle&) = delete;        // prevent double-close
    FileHandle& operator=(const FileHandle&) = delete;
private:
    std::FILE* fp_;
};
```

RAII replaces manual `acquire()` / `release()` pairs — and the try/finally blocks other languages need — with ordinary scope rules. `std::unique_ptr`, `std::lock_guard`, and `std::vector` are all RAII types under the hood.

Why asked: Considered table-stakes for any C++ role above entry level; a wrong or vague answer here is a strong negative signal.

Q10. Explain the Rule of Three, Rule of Five, and Rule of Zero.

INTERMEDIATE

DEFINITION

Rule of Three: if a class needs a user-defined destructor, copy constructor, or copy-assignment operator, it almost certainly needs all three. **Rule of Five** (C++11+) extends this to move constructor and move-assignment operator. **Rule of Zero:** prefer designing classes that need *none* of the five, by delegating ownership entirely to members that already manage themselves (smart pointers, containers, strings).

```
// Rule of Five – this class owns a raw resource, so all five are needed
class Buffer {
public:
    explicit Buffer(size_t n) : size_(n), data_(new int[n]) {}
    ~Buffer() { delete[] data_; } // 1. destructor
    Buffer(const Buffer& o) : size_(o.size_), data_(new int[o.size_]) {
        std::copy(o.data_, o.data_ + size_, data_);
    } // 2. copy ctor
    Buffer& operator=(const Buffer& o) { Buffer tmp(o); swap(tmp); return *this; } // 3. copy assign
    Buffer(Buffer&& o) noexcept : size_(o.size_), data_(o.data_) {
        o.data_ = nullptr; o.size_ = 0;
    } // 4. move ctor
    Buffer& operator=(Buffer&& o) noexcept { swap(o); return *this; } // 5. move assign
    void swap(Buffer& o) noexcept { std::swap(size_, o.size_); std::swap(data_, o.data_); }
private:
    size_t size_; int* data_;
};

// Rule of Zero – equivalent ownership, no hand-written special members at all
class SafeBuffer {
public:
    explicit SafeBuffer(size_t n) : data_(std::make_unique<int[]>(n)) {}
private:
    std::unique_ptr<int[]> data_; // unique_ptr already does the Rule of Five correctly
};
```

Modern C++ style strongly prefers Rule of Zero: every hand-written special member function is a place a leak, double-free, or slicing bug can hide.

Why asked: One of the single most common senior C++ interview questions — it tests memory-safety instincts more than syntax.

Q11. Why must a base class designed for polymorphic use declare a `virtual` destructor?

ADVANCED

```
struct Base {
    ~Base() { std::puts("Base cleanup"); }    // NOT virtual – the bug
};
struct Derived : Base {
    Derived() : resource_(new int[100]) {}
    ~Derived() { delete[] resource_; std::puts("Derived cleanup"); }
private:
    int* resource_;
};

Base* p = new Derived();
delete p; // UB: only ~Base() runs. Derived::resource_ leaks, and Derived's
          // destructor body never executes – the behavior is undefined per the standard.
```

Without a virtual destructor, `delete` on a base pointer resolves the destructor call statically — using the pointer's declared type, not the object's dynamic type — so any derived-class cleanup is silently skipped. Meyers' *Effective C++* Item 7 states the rule directly: "Polymorphic base classes should declare virtual destructors. If a class has any virtual functions, it should have a virtual destructor." Conversely, giving a virtual destructor to a class never meant to be a polymorphic base adds needless vtable overhead — so the rule cuts both ways.

Why asked: Extremely common "what's wrong with this code" whiteboard prompt — a fast, unambiguous way to test real understanding of dynamic dispatch versus static typing.

Q12. Is it safe to call a virtual function from inside a constructor or destructor?

ADVANCED

```
struct Base {
    Base() { init(); }    // calls Base::init(), NEVER Derived::init()
    virtual void init() { std::puts("Base::init"); }
};
struct Derived : Base {
    void init() override { std::puts("Derived::init"); }
};
Derived d; // prints "Base::init" – not the override you might expect
```

It compiles and runs, but the result surprises most candidates: during a base class's own constructor, the object's dynamic type *is* the base class — the derived part of the object hasn't been constructed yet, so a virtual call resolves to the base's implementation, never a more-derived override. The symmetric rule applies during destructors, in reverse order. Meyers' *Effective C++* Item 9 codifies this: don't call virtual functions during construction or destruction, because such calls will never go to a more derived class than the one currently executing.

Why asked: A favorite "explain the output" trap question precisely because the compiler gives no warning — it's a design footgun, not a syntax error.

04 Polymorphism & Dynamic Dispatch Internals

This is the deepest and most differentiating category in a C++ OOP loop. Interviewers use it to distinguish candidates who *use* virtual functions from candidates who understand the vtable/vptr machinery, the cost model, and the sharp edges — slicing, binding time, and abstract-class rules.

Q13. How are virtual functions implemented under the hood? Explain the vptr and the vtable.

ADVANCED

Most compilers implement virtual dispatch with two structures: a per-*class* **virtual table (vtable)** — a static array of function pointers, one slot per virtual function — and a per-*object* hidden **virtual pointer (vptr)**, set by the constructor to point at its class's vtable. Every object of the same class shares one vtable; only the vptr lives inside each object's memory layout.

```
struct Shape {
    virtual double area() const { return 0; }
    virtual void draw() const {}
    virtual ~Shape() = default;
};

struct Circle : Shape {
    double area() const override { return 3.14 * r_ * r_; }
    void draw() const override { /* ... */ }
    double r_ = 1.0;
};

// Conceptual memory layout of a Circle object:
// [ vptr ] -----> Circle's vtable: [ &Circle::area, &Circle::draw, &Circle::~Circle ]
// [ r_   ]
//
// Shape* s = new Circle();
// s->area(); → 1) follow s's vptr to Circle's vtable
//             2) index the "area" slot (same index in every class in the hierarchy)
//             3) call the function pointer found there
```

This is why dynamic dispatch has real, bounded cost — one extra pointer dereference and an indirect call versus a direct call — and why every polymorphic object carries the memory overhead of one vptr (typically 8 bytes on a 64-bit system), regardless of how many virtual functions the class declares.

Why asked: The single most-cited "advanced C++" interview question across LeetCode Discuss threads and systems-programming loops; it separates candidates who can reason about cost from those who only know the keyword.

Q14. What is a pure virtual function, and how does that differ from a virtual function with an empty body? What is an "abstract class" versus an "interface" in C++?

```
class Drawable {
public:
    virtual void draw() const = 0;    // PURE virtual – no implementation required
    virtual ~Drawable() = default;
};

// Drawable d; // ERROR: cannot instantiate an abstract class
```

A virtual function with a body (even an empty one, `virtual void f() {}`) still provides a usable default — the class remains instantiable. A **pure** virtual function (`= 0`) provides no implementation and forces every concrete (non-abstract) derived class to override it, or that derived class is itself abstract too. Any class with at least one pure virtual function is an **abstract class**: it cannot be instantiated but can be used polymorphically through pointers/references. C++ has no dedicated `interface` keyword; by convention, a class whose *every* member function is pure virtual (and which typically has no data members) is called an "interface class" — the idiom that stands in for Java's `interface`.

Why asked: Tests precision of vocabulary — many candidates conflate "abstract class" and "interface" because other languages make them the same keyword.

Q15. What is object slicing? Show how it happens and how to prevent it.

```
struct Animal { virtual std::string speak() const { return "..."; } };
struct Dog : Animal { std::string speak() const override { return "Woof"; } };

void report(Animal a) { std::cout << a.speak(); } // takes Animal BY VALUE

Dog d;
report(d); // prints "..." – SLICED. `a` is a true Animal, Dog part is discarded

void reportRef(const Animal& a) { std::cout << a.speak(); }
reportRef(d); // prints "Woof" – no slicing, dynamic type preserved through reference
```

Object slicing occurs when a derived-class object is copied (or passed/returned) into a base-class object *by value*. Only the base subobject is copied; every derived-only data member and the object's true dynamic type are lost, so subsequent virtual calls on the sliced copy dispatch using the base class's implementation. The fix is structural, not clever: always pass and store polymorphic types through pointers or references (raw, or better, smart pointers), never by value.

Why asked: A staple "spot the bug" interview question because it produces silently wrong behavior with zero compiler warning.

Q16. Distinguish early (static) binding from late (dynamic) binding in C++.

FOUNDATIONAL

DEFINITION

Early binding resolves a function call to a specific address at compile time — used for non-virtual functions, free functions, and overload resolution. Late binding defers that resolution to runtime, using the object's actual dynamic type — used exclusively for calls to virtual functions made through a base pointer or reference.

```
struct Base { void nonVirtual() { std::puts("Base::nonVirtual"); }  
              virtual void virtualFn() { std::puts("Base::virtualFn"); } };  
struct Derived : Base { void nonVirtual() { std::puts("Derived::nonVirtual"); }  
                        void virtualFn() override { std::puts("Derived::virtualFn"); } };  
  
Base* b = new Derived();  
b->nonVirtual(); // "Base::nonVirtual"    - early binding: resolved by pointer's static type  
b->virtualFn();  // "Derived::virtualFn"  - late binding: resolved by object's dynamic type
```

Note the trap in the first call: `nonVirtual()` is *hidden*, not overridden, by `Derived::nonVirtual`, so calling it through a `Base*` always reaches `Base`'s version regardless of what the pointer actually points to.

Why asked: Frequently paired with Q15 (slicing) as a two-part question testing whether you can predict runtime behavior purely by reading declarations.

Q17. Can a constructor be virtual? If not, how do you achieve "virtual construction"?

ADVANCED

No — a constructor cannot be virtual. Virtual dispatch requires a vptr that is only set up *during* construction (once the compiler knows which constructor is running), so there is no valid vtable to dispatch through before an object exists. The standard workaround is the **virtual clone / factory pattern**: a regular virtual member function that internally knows how to construct a new object of its own concrete type.

```
struct Shape {
    virtual std::unique_ptr<Shape> clone() const = 0;
    virtual ~Shape() = default;
};
struct Circle : Shape {
    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Circle>(*this); // knows its own concrete type
    }
};

void duplicate(const Shape& s) {
    std::unique_ptr<Shape> copy = s.clone(); // "virtual copy construction"
}
```

Destructors, by contrast, absolutely can and — per Q11 — often should be virtual, since the object fully exists and has a valid vptr at the point of destruction.

Why asked: Tests whether you understand vptr initialization order well enough to explain a hard "no" rather than guessing.

05 Inheritance Complexities: Multiple Inheritance & the Diamond Problem

C++ is one of the few mainstream OOP languages that permits true multiple inheritance, and interviewers treat that as a signal-rich area: it exposes whether you understand memory layout, ambiguity resolution, and — critically — when *not* to reach for inheritance at all.

Q18. What is the diamond problem, and how does virtual inheritance resolve it?

ADVANCED

```
struct Animal      { int legs = 4; };
struct Bird   : Animal {};
struct Bat    : Animal {};
struct Batbird : Bird, Bat {};    // diamond: Animal reached via TWO paths

Batbird b;
// b.legs;           // ERROR: ambiguous — Bird::Animal::legs vs Bat::Animal::legs
b.Bird::legs = 2;     // must disambiguate explicitly — and the two copies are independent

// --- Fix: virtual inheritance ---
struct VAnimal      { int legs = 4; };
struct VBird : virtual VAnimal {};
struct VBat  : virtual VAnimal {};
struct VBatbird : VBird, VBat {};

VBatbird vb;
vb.legs = 2;    // OK — exactly one shared VAnimal subobject exists
```

Without `virtual`, `Batbird` contains two independent `Animal` subobjects — one via `Bird`, one via `Bat` — so any unqualified reference to a common base member is ambiguous, and the two copies can silently drift out of sync. Declaring the inheritance `virtual public VAnimal` in both intermediate classes tells the compiler to allocate exactly one shared base subobject, constructed once by the most-derived class regardless of how many paths lead to it.

Why asked: Probably the single most iconic C++-specific OOP interview question — it doesn't exist in single-inheritance languages, so it's a fast way to confirm real C++ depth.

Q19. What costs does virtual inheritance introduce, beyond fixing the diamond?

ADVANCED

Virtual inheritance is not free: objects using it typically grow by an extra pointer (or offset) per virtual base to locate the shared subobject at runtime, since its position can no longer be a fixed compile-time offset (it depends on the most-derived type). Construction order also gets subtler — the **most-derived class** becomes directly responsible for initializing the virtual base, bypassing the intermediate classes' own attempts to initialize it, which surprises engineers who expect each class to fully own its immediate base's construction.

Why asked: A natural, higher-bar follow-up to Q18 that filters candidates who memorized the fix from those who understand its trade-offs.

Q20. "Favor composition over inheritance"

INTERMEDIATE

— justify this with a C++ example.

```
// Inheritance (is-a) – tempting but wrong: a Car is not a kind of Engine
class Car : public Engine { };    // exposes Engine's whole public interface as Car's own

// Composition (has-a) – Car owns an Engine and exposes only what it chooses to
class Engine {
public:
    void start() { /* ... */ }
};
class Car {
public:
    void start() { engine_.start(); }    // selective delegation, engine_ stays hidden
private:
    Engine engine_;
};
```

Inheritance should model a genuine "is-a, substitutable-for" relationship (a `Circle` really is a `Shape` everywhere a `Shape` is expected — the Liskov Substitution Principle). Composition models "has-a" and is preferred by default because it: (1) doesn't leak the whole base interface to callers, (2) avoids fragile-base-class problems where a base class change silently breaks every derived class, and (3) can be swapped or reconfigured at runtime by holding a pointer/interface instead of a fixed compile-time base.

Why asked: A design-judgment question common in senior and staff loops — it's really asking whether you'll reach for inheritance reflexively or deliberately.

06 Memory Management & Smart Pointers

C++ OOP interviews treat ownership modeling as an extension of encapsulation: a well-designed class should make its ownership of resources explicit and automatic. Smart pointers are the modern, idiomatic vocabulary for that, and questions here probe both API familiarity and the underlying cost model.

Q21. Compare `unique_ptr` , `shared_ptr` , and `weak_ptr` — ownership model, cost, and when to choose each.

FOUNDATIONAL

SMART POINTER	OWNERSHIP	OVERHEAD	USE WHEN
<code>unique_ptr<T></code>	Exclusive, single owner	Same size as a raw pointer; move-only	The default choice — one clear owner
<code>shared_ptr<T></code>	Shared, reference-counted	Extra control block; atomic increment/decrement	Genuinely shared, lifetime-uncertain ownership
<code>weak_ptr<T></code>	Non-owning observer of a <code>shared_ptr</code>	Doesn't affect the strong reference count	Breaking cycles; caches; "does this still exist?"

```
auto owner = std::make_unique<Widget>();           // exclusive owner
auto shared1 = std::make_shared<Widget>();         // refcount = 1
auto shared2 = shared1;                            // refcount = 2, both own it
std::weak_ptr<Widget> observer = shared1;          // refcount still 2

if (auto locked = observer.lock()) {                // must "lock" to safely use it
    locked->doSomething();
}
```

Prefer `unique_ptr` by default — it has zero overhead versus a raw pointer and makes ownership unambiguous at every call site (passing it means transferring ownership via `std::move`). Reach for `shared_ptr` only when lifetime genuinely can't be tied to one owner (e.g., a node referenced from multiple independent graphs). `weak_ptr` exists specifically to observe a `shared_ptr` -managed object without extending its lifetime or creating a cycle.

Why asked: One of the highest-frequency modern-C++ questions; interviewers are listening for the ownership reasoning, not just the API names.

Q22. What is a `shared_ptr`'s "control block," and why is `make_shared` preferred over `shared_ptr<T>(new T)` ?

ADVANCED

Every `shared_ptr` group shares one heap-allocated **control block** holding the strong reference count, the weak reference count, and (often) a type-erased deleter. Writing `std::shared_ptr<Widget>(new Widget())` performs *two* separate heap allocations — one for the `Widget`, one for the control block — and if the control-block allocation throws after the `Widget` allocation already succeeded, a leak is possible in older/unusual code paths.

`std::make_shared<Widget>()` performs a **single** allocation that lays the object and its control block out together, which is both faster and safer.

Why asked: A precise, "have you actually used this in production" question that's hard to fake.

Q23. How can `shared_ptr` cause a memory leak despite being a "smart" pointer, and how does `weak_ptr` fix it?

ADVANCED

```
struct Node {
    std::shared_ptr<Node> next;
    std::shared_ptr<Node> prev;    // PROBLEM: creates a reference cycle
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;
b->prev = a;    // a and b now hold each other alive - refcount never reaches 0, leak

// Fix: break the cycle with weak_ptr on the "back" reference
struct SafeNode {
    std::shared_ptr<SafeNode> next;
    std::weak_ptr<SafeNode> prev;    // doesn't keep the owner alive
};
```

Reference counting only reclaims memory when the count reaches zero. Two objects holding `shared_ptr`s to each other keep each other's count above zero forever, even if nothing external references either — a classic reference cycle leak. Replacing the "back-pointer" direction with `weak_ptr` preserves navigability (via `.lock()`) without contributing to the strong count, so the cycle can actually be collected.

Why asked: Confirms you understand reference counting's one structural weakness, which is invisible until it causes a production leak.

Q24. What is the pImpl (pointer-to-implementation) idiom, and what problem does it solve?

ADVANCED

```
// widget.h - public header, ships to every consumer
class Widget {
public:
    Widget();
    ~Widget();
    void doWork();
private:
    class Impl; // forward declaration only
    std::unique_ptr<Impl> pImpl_; // one pointer - header never changes when Impl does
};

// widget.cpp - implementation, private to this translation unit
class Widget::Impl {
public:
    void doWork() { /* all the messy real logic + private data lives here */ }
};
Widget::Widget() : pImpl_(std::make_unique<Impl>()) {}
Widget::~~Widget() = default;
void Widget::doWork() { pImpl_>doWork(); }
```

pImpl moves a class's private data members and helper logic behind a single opaque pointer to a forward-declared implementation class. Consumers who `#include "widget.h"` never see `Impl`'s definition, so changing private implementation details no longer forces every dependent file to recompile — a major win for build times in large codebases — and it also creates a genuine binary/ABI-stability boundary. The cost is one extra heap allocation and one pointer indirection per access, plus (as raised by the GameDeveloper/bitsquid comparisons) it is a compilation firewall for a single concrete type, not a substitute for a polymorphic interface when you actually need multiple interchangeable implementations.

Why asked: A staple large-codebase / build-hygiene question in senior interviews, especially at companies with big monorepos.

07 Move Semantics & Modern C++ (C++11 and Later)

Since C++11, "OOP interview" questions routinely blend into modern-C++ questions, because move semantics change how special member functions — a core OOP object-lifecycle topic — are written and reasoned about.

Q25. What is an rvalue reference, and how does it enable move semantics?

INTERMEDIATE

DEFINITION

An lvalue is an expression referring to an object with persistent identity (you can take its address); an rvalue is a temporary with no persistent identity beyond the current expression. `T&&` is an rvalue reference — it binds only to rvalues, and signals to the compiler "this argument is a temporary; it's safe to cannibalize its resources instead of copying them."

```
std::vector<int> makeVec() { return std::vector<int>(1'000'000, 0); }

std::vector<int> v = makeVec(); // pre-C++11 semantics: copies 1M ints. With move semantics:
                               // the temporary returned by makeVec() is an rvalue, so the
                               // MOVE constructor runs instead — it steals the internal
                               // heap buffer pointer in O(1), no element copy at all.
```

Why asked: Foundational modern-C++ knowledge; almost every mid-to-senior loop touches this.

Q26. Implement a move constructor and move-assignment operator for a class that owns a raw resource, and explain the state they must leave the source in.

INTERMEDIATE

```
class Buffer {
public:
    Buffer(Buffer&& other) noexcept
        : data_(other.data_), size_(other.size_) {
        other.data_ = nullptr;    // source left in a VALID, destructible, empty state
        other.size_ = 0;
    }
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;        // release whatever *this* currently owns
            data_ = other.data_;    // steal the resource
            size_ = other.size_;
            other.data_ = nullptr;  // leave source safely destructible
            other.size_ = 0;
        }
        return *this;
    }
    ~Buffer() { delete[] data_; }
private:
    int* data_; size_t size_;
};
```

The moved-from object must remain in a **valid but unspecified state** — its destructor and any subsequent assignment to it must still work correctly, even though its old value is gone. Move operations should also be marked `noexcept` whenever possible: standard containers like `std::vector` check this at compile time and will silently fall back to copying during reallocation if the move constructor could throw, defeating the entire optimization.

Why asked: A hands-on coding question, not just a definitions question — tests whether you can actually write correct Rule-of-Five code under pressure.

Q27. What does `std::move` actually do?
How does it differ from `std::forward`?

INTERMEDIATE

`std::move` moves nothing by itself — it is purely a **cast** to an rvalue reference (`static_cast<T&&>`). It relabels an lvalue as "moveable," which makes overload resolution prefer a move constructor/assignment over a copy — the actual resource transfer happens inside whatever move constructor gets selected. `std::forward` is conditional: inside a template, it preserves the original value category of a forwarding reference — an argument that was passed as an lvalue stays an lvalue, one passed as an rvalue stays an rvalue — which is what makes "perfect forwarding" perfect.

```
template <typename T, typename... Args>
std::unique_ptr<T> make(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...)); // preserves each arg's value category
}
```

Why asked: A classic "explain the difference" question that trips up candidates who use both interchangeably without understanding why `forward` is template-specific.

08 Operator Overloading & Special Member Functions

Operator overloading is where OOP syntax and C++'s value semantics intersect most visibly – and where subtle correctness bugs (self-assignment, exception safety) are most commonly probed.

Q28. Why must a hand-written `operator=` handle self-assignment, and what's the cleanest way to guarantee it?

INTERMEDIATE

```
class Buffer {
public:
    // Copy-and-swap idiom: self-assignment-safe AND strongly exception-safe
    Buffer& operator=(Buffer other) {    // note: takes by VALUE, so a copy already happened
        swap(*this, other);            // swap with the temporary – never touches raw pointers unsafely
        return *this;
    }
    // `other` (holding our OLD data) is destroyed here, cleanly
    friend void swap(Buffer& a, Buffer& b) noexcept {
        std::swap(a.data_, b.data_); std::swap(a.size_, b.size_);
    }
private:
    int* data_; size_t size_;
};
```

A naive `operator=` that does `delete[] data_; data_ = new int[other.size_]; ...` destroys its own data before checking whether `other` is `*this` – under self-assignment (`a = a;`, or indirectly through aliased references), it frees the very memory it's about to read from. The **copy-and-swap idiom** sidesteps the problem entirely rather than patching it with an `if (this != &other)` guard: it builds the new state fully in a temporary first (exception safety: if that copy throws, `*this` is untouched), then atomically swaps it in with a `noexcept` swap.

Why asked: Tests whether you reach for the idiomatic, provably-correct pattern versus an ad-hoc guard that's easy to get subtly wrong elsewhere in a class hierarchy.

Q29. When overloading a binary operator like `operator==`, should it be a member function or a free (non-member) function?

INTERMEDIATE

```
class Money {
public:
    explicit Money(long cents) : cents_(cents) {}
    long cents() const { return cents_; }
private:
    long cents_;
};
// Free function: allows implicit conversions on BOTH sides, e.g. `5 == m` and `m == 5`
bool operator==(const Money& a, const Money& b) { return a.cents() == b.cents(); }
```

Operators that must modify their left operand (`operator=`, `operator+=`, `operator[]`) should be members, since they inherently need a specific receiver object. Symmetric operators (`=`, `+`, `<<`) are generally better as free functions (often `friend` s, per Q7) so that implicit conversions apply uniformly to *both* operands – a member `operator==` only allows conversions on the right-hand argument, silently making the operator asymmetric.

Why asked: A design-nuance question that separates candidates who've only overloaded operators casually from those who've hit the asymmetric-conversion bug in practice.

Q30. Why do you typically need two overloads of `operator[]` – one `const`, one not?

FOUNDATIONAL

```
class Row {
public:
    int& operator[](size_t i) { return data_[i]; } // mutable access
    const int& operator[](size_t i) const { return data_[i]; } // read-only access on const objects
private:
    std::array<int, 8> data_{};
};
```

Overload resolution picks the `const` version automatically whenever the object itself is `const` (or accessed through a `const` reference/pointer), returning a `const int&` that prevents mutation. Providing only the non-`const` version would make the class unusable through any `const Row&` parameter; providing only the `const` version would prevent writing through `row[i] = 5;` on a mutable object. This "const-overload pair" pattern generalizes to any accessor, not just `operator[]`.

Why asked: Common in live-coding rounds where you implement a small container class from scratch.

09 Whiteboard Rapid-Fire

Fast, single-answer questions used to open a loop or fill the last five minutes. Expect these to be asked verbally with no code in front of you.

Can a constructor be private? Why would you do that?	Yes — used to force object creation through a static factory method or to implement the Singleton pattern, preventing direct instantiation.
Can you overload the destructor?	No. A class has exactly one destructor; it takes no arguments and cannot be overloaded.
What is the size of an empty class in C++, and why isn't it zero?	1 byte (implementation-defined minimum). The standard guarantees every object has a unique address, so two distinct empty-class instances must occupy distinct addresses — impossible with zero-sized objects in an array.
Can a static member function be declared virtual?	No — virtual dispatch resolves through an object's vptr, and static functions have no associated object (no <code>this</code>), so there is nothing to dispatch on.
Can you call a pure virtual function at all?	Yes, if you provide an out-of-class definition and call it with explicit qualification, e.g. <code>Base::pureVirtualFn()</code> , most commonly from within a derived class's own override, or from the base's destructor.
Function overloading vs. overriding — one-line distinction?	Overloading: same name, different parameter list, resolved at compile time, no inheritance required. Overriding: same signature, redefined in a derived class, resolved at runtime via <code>virtual</code> .
What does "hiding" mean in the context of inheritance?	A derived-class member function with the same name as a base one (but a different, non-overriding signature, or non-virtual) hides ALL overloads of that name in the base — they become inaccessible without <code>Base::name</code> qualification or a <code>using</code> declaration.
Does every polymorphic object pay for a vptr per virtual function?	No — one vptr per object total, regardless of how many virtual functions the class has; the vtable itself (one per class, not per object) holds one slot per virtual function.
What's the default access level of inheritance when using the <code>class</code> keyword with no specifier?	<code>private</code> — <code>class D : Base {}</code> means private inheritance; <code>struct D : Base {}</code> defaults to public inheritance.
Is <code>final</code> on a class or virtual function an OOP feature?	Yes (C++11) — <code>final</code> on a class prevents further inheritance; on a virtual function it prevents further overriding, which also lets the compiler devirtualize (skip vtable lookup) that call in some cases.

This guide's structure and question set were shaped by live web research across community and academic references rather than a fixed template, per the following sources:

- [GeeksforGeeks — C++ OOP Interview Questions](#)
- [GeeksforGeeks — 30 OOPs Interview Questions](#)
- [GeeksforGeeks — Top C++ Interview Questions](#)
- [GeeksforGeeks — vTable and vPtr in C++](#)
- [GeeksforGeeks — Diamond Problem in C++](#)
- [GeeksforGeeks — Smart Pointer Comparison](#)
- [LeetCode Discuss — Virtual Function in C++](#)
- [LeetCode Discuss — Top 16 OOPs Interview Questions](#)
- [Glassdoor — Object-Oriented Programming Interview Questions](#)
- [Glassdoor — Jump Trading C++ Interview Report](#)
- [Wikipedia — Rule of Three / Five \(C++\)](#)
- [Wikipedia — Virtual Inheritance](#)
- [Effective C++ \(Scott Meyers\) — Item Summary](#)
- [Game Developer — PIMPL vs. Pure Virtual Interfaces](#)
- [Herb Sutter — GotW #100: Compilation Firewalls](#)
- [techinterview.org — C++ Interview Questions 2025](#)
- [Techie Delight — Static vs. Dynamic Binding](#)

Compiled September 2026 · Standard reference: ISO C++ (C++11–C++20) · For personal interview preparation.